Build REST APIs for scientific data

A four-hour lab with Django REST Framework

From running the app to three working endpoints

The four-hour plan

Time	Activity
09:00–09:15	Short introduction
09:15–09:35	0. Clone the workshop branch
09:35–10:00	1. Start the application
10:00–10:15	2. Inspect the existing API
10:15–10:50	3. Easy: metacluster summary
10:50–11:00	Break
11:00–11:45	4. Medium: filtered structural search
11:45–12:45	5. Hard: protein domain annotations
12:45–13:00	6. Test and demonstrate

What is an API?

- An **Application Programming Interface** defines how software can request data or actions from other software.
- A **library API** is called within your program, such as a NumPy function.
- A **web API** receives requests over a network. A browser, `curl`, or a Python script can be its client.

Checkpoint Our client sends an HTTP request; Django sends an HTTP response.

Common web API designs

Name	What it is	Main idea
REST	Architectural style	Resources have URLs; HTTP methods express operations.
GraphQL	Query language	The client specifies which fields it wants.
gRPC	RPC framework	The client calls defined remote methods with typed messages.
SOAP	Protocol	Exchange structured XML messages.

These are different kinds of technology, not four interchangeable protocols. This workshop uses REST-style HTTP endpoints.

REST: one request, one response

```
GET /api/dpcfam/mcs/MC1/ HTTP/1.1
Accept: application/json

HTTP/1.1 200 OK
Content-Type: application/json

{"mcid": "MC1", "size_uniref50": 260, "pfam_da": "PF00001"}
```

- The URL identifies a resource; **GET** requests its representation.
- The response has a **status**, **headers**, and a **body**.
- JSON is our representation format; REST can use other formats.
- REST is **stateless**: each request carries the information needed to process it.

HTTP methods and status codes

- **GET**: read; **POST**: commonly create or submit an action.
- **PUT/PATCH**: update; **DELETE**: remove.
- Our three endpoints are **read-only**: they accept GET.

Status	Meaning in this workshop
200	The request succeeded.
400	A query parameter is invalid.
404	The requested cluster or protein does not exist.
405	The HTTP method is not allowed, such as POST to a GET-only view.

What you will learn

1. Read one DPCfam metacluster.
2. Filter DPCstruct clusters by size and confidence.
3. Combine protein mappings from DPCfam, DPCstruct, and Pfam.

Request $\rightarrow$ URL route $\rightarrow$ view $\rightarrow$ Django model query $\rightarrow$ JSON response.

Before the workshop starts

- You need **Git, Python 3.12+, curl, Docker with Compose**, and a code editor.
- Docker must be running, and your user must be able to access it.
- Follow `workshop/README.md` on the workshop branch.
- Open **two terminals**: one for Django, one for requests.

Checkpoint The setup generates local credentials; no instructor-provided `.env` is needed.

The small teaching database

- **Synthetic data**: invented for learning, not biological evidence.
- DPCfam and DPCstruct each contain `MC1`–`MC40`. An ID such as `MC100` is absent.
- `WSDEMO0001` has 130 mappings in each annotation table.
- `WSEMPTY0001` exists with no mappings; `WSMISSING` does not exist.

Checkpoint Use the workshop setup. Large structure and sequence downloads are not needed for these API exercises.

0. Clone and find the Django files

1. Check Python, Git, and Docker Compose.
2. Clone branch `workshop/student-setup` into a new student folder.
3. Locate `manage.py`, `api/urls.py`, and `dpcfam/models.py`.

Checkpoint You are in `dpc_workshop_student` and can open `workshop/README.md`.

0. Solution: clone and inspect

```
python3 --version
git --version
docker compose version
docker info --format '{{.ServerVersion}}'

REPO=https://github.com/RitAreaSciencePark
git clone --branch workshop/student-setup \
  "$REPO/dpc_fam_and_struct_webapp.git" dpc_workshop_student
cd dpc_workshop_student
ls manage.py api/urls.py dpcfam/models.py
```

`api/urls.py` connects API URLs to views.

`models.py` describes records and their relationships.

1. Run DPCexplorer

1. Create a virtual environment; install `workshop/requirements.txt`.
2. Run `workshop/setup.py`: create the small database, load its data, run migrations, and verify the app.
3. Start Django using `workshop/manage.py` on port `8001`.

Checkpoint Open `http://127.0.0.1:8001/`. Keep this port in every request.

1. Solution: start the server

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r workshop/requirements.txt

python workshop/setup.py
python workshop/manage.py runserver 8001
```

Keep this terminal open. Use terminal B for API calls.

The wrapper selects `.env.workshop`. The database uses port 55432; Django uses 8001. If 8001 is busy, select another free port and update your URLs.

2. Explore the API that already exists

1. Request DPCfam metacluster `MC1`.
2. Request DPCstruct metacluster `MC5`; find `plddt`.
3. Request three members of DPCfam `MC1`.
4. Find `count`, `results`, and `next`; request the next page.

Checkpoint Distinguish the total member count from the number of members returned on one page.

2. Solution: make three GET requests

```
ROOT=http://127.0.0.1:8001/api
curl -i "$ROOT/dpcfam/mcs/MC1/"
curl -i "$ROOT/dpcstruct/mcs/MC5/"
curl -i "$ROOT/dpcfam/mcs/MC1/members/?page_size=3"
```

- `-i` displays the status and headers before the JSON body.
- DPCfam `MC1`: `size_uniref50 = 260`.
- DPCstruct `MC5`: `plddt = 70.0`, a synthetic confidence value.
- Quote the URL to keep query parameters together, especially `&`.

2. Solution: follow the next page

Field	First page of this teaching dataset
`count`	260 members in total.
`results`	Three member objects, with IDs 1, 2, and 3.
`next`	A URL containing `page=2&page_size=3`.
`previous`	`null`, because this is the first page.

```
curl "$ROOT/dpcfam/mcs/MC1/members/?page=2&page_size=3"
```

Page 2 returns IDs 4, 5, and 6. Its `next` URL points to page 3. Follow the URL returned by the API.

3. Easy: summarize one metacluster

GET `/api/workshop/dpcfam/MC1/summary/`

1. **Create** `api/workshop_views.py`; it is not in the starting project.
2. Write a GET-only view that finds the cluster by `mcid`.
3. Return `mcid`, `size_uniref50`, and `pfam_da`.
4. Add its route to the existing `api/urls.py`.

Checkpoint Known ID $\rightarrow$ 200; missing ID $\rightarrow$ 404; POST $\rightarrow$ 405.

3. Solution: import the tools

New file: `api/workshop_views.py`

```
from django.shortcuts import get_object_or_404
from rest_framework.decorators import api_view
from rest_framework.response import Response
from dpcfam.models import DpcfamMcsProperty
```

- `get_object_or_404`: retrieve a record or return HTTP 404.
- `api_view`: make the function a DRF view with allowed methods.
- `Response`: return data that DRF can render as JSON.
- `DpcfamMcsProperty`: query the cluster-property table.

3. Solution: write the summary view

```
@api_view(["GET"])
def dpcfam_summary(request, mcid):
    mc = get_object_or_404(DpcfamMcsProperty, mcid=mcid)
    return Response({
        "mcid": mc.mcid,
        "size_uniref50": mc.size_uniref50,
        "pfam_da": mc.pfam_da,
    })
```

- In `mcid=mcid`, the left side is the model field; the right side is the function argument captured from the URL.
- The dictionary selects the three response fields.
- The decorator rejects POST with 405; the lookup handles missing IDs with 404.

3. Solution: connect the URL

In `api/urls.py`, add the import:

```
from .workshop_views import dpcfam_summary
```

Add this entry inside the existing `urlpatterns` list:

```
path(
    "workshop/dpcfam/<str:mcid>/summary/",
    dpcfam_summary,
    name="workshop-dpcfam-summary",
),
```

Keep the existing routes. The project already adds `/api/`; `<str:mcid>` passes the URL value to your function.

3. Solution: test success and failure

```
BASE=http://127.0.0.1:8001/api/workshop

# Expected: 200
curl -i "$BASE/dpcfam/MC1/summary/"
# Expected: 404
curl -i "$BASE/dpcfam/MC999999/summary/"
# Expected: 405
curl -i -X POST "$BASE/dpcfam/MC1/summary/"
```

A known cluster returns the three requested fields. An error should return a deliberate status and a clear message, not a server traceback.

4. Medium: search structural clusters

GET `/api/workshop/dpcstruct/search/`

1. Read `min_plddt` (default 0) and `min_size` (default 1).
2. Require a numeric pLDDT in $[0, 100]$ and an integer size ≥ 1; reject invalid input with 400.
3. Filter on `plddt` and `mc_size`; order by highest pLDDT, then cluster ID.
4. Return a JSON list of at most ten objects: `mc_id`, `mc_size`, `plddt`.

Checkpoint `?min_plddt=80&min_size=10` succeeds; `min_plddt=120` returns 400.

4. Solution: read the parameters

Add the model import, then append the new function.

```
from dpcstruct.models import DpcStructMcsProperty

@api_view(["GET"])
def dpcstruct_search(request):
    min_plddt = request.query_params.get("min_plddt", "0")
    min_size = request.query_params.get("min_size", "1")
```

- `request.query_params` reads values after `?` in the URL.
- `.get(...)` supplies the default if a parameter is absent.
- These values are text; convert and validate them before querying.

4. Solution: convert text to numbers

Continue inside `dpcstruct_search()`:

```
    try:
        min_plddt = float(min_plddt)
        min_size = int(min_size)
    except ValueError:
        return Response(
            {"error": "Use a numeric pLDDT and an integer size."},
            status=400,
        )
```

Query parameters arrive as text. `float()` permits decimals; `int()` requires a whole-number string.

4. Solution: validate the range

Continue inside `dpcstruct_search()`, aligned with `try`:

```
    if not 0 <= min_plddt <= 100 or min_size < 1:
        return Response(
            {"error": "Use pLDDT 0-100 and size at least 1."},
            status=400,
        )
```

- `hello` fails numeric conversion.
- 120 converts successfully, but fails the pLDDT range check.
- `min_size=0` fails the minimum-size check.

Checkpoint Validate before querying the database. Invalid input returns 400.

4. Solution: filter and order

Continue inside `dpcstruct_search()`:

```
    matches = (
        DpcStructMcsProperty.objects.filter(
            plddt__gte=min_plddt,
            mc_size__gte=min_size,
        )
        .order_by("-plddt", "mc_id")[:10]
    )
```

- `__gte`: greater than or equal to the supplied minimum.
- `-plddt`: descending confidence; `mc_id` breaks ties.
- `[:10]`: ask the database for at most ten records.

4. Solution: return a JSON list

Finish `dpcstruct_search()`:

```
    return Response([
        {
            "mc_id": mc.mc_id,
            "mc_size": mc.mc_size,
            "plddt": mc.plddt,
        }
        for mc in matches
    ])
```

The list comprehension builds one dictionary per match. No matches gives `[]` with HTTP 200. This endpoint returns a list directly, not a `results` wrapper.

4. Solution: register the search

Extend the workshop import in `api/urls.py`:

```
from .workshop_views import dpcfam_summary, dpcstruct_search
```

Add inside the existing `urlpatterns`:

```
path(
    "workshop/dpcstruct/search/",
    dpcstruct_search,
    name="workshop-dpcstruct-search",
),
```

Save both files. Django's development server reloads the code; watch terminal A for import or indentation errors.

4. Solution: test the filters

```
BASE=http://127.0.0.1:8001/api/workshop

# Expected: 200, ten matches in this dataset
curl -i "$BASE/dpcstruct/search/?min_plddt=80&min_size=10"
# Expected: 400, invalid range
curl -i "$BASE/dpcstruct/search/?min_plddt=120"
# Expected: 400, invalid numeric input
curl -i "$BASE/dpcstruct/search/?min_size=hello"
```

For the valid request, check every size is at least 10, every pLDDT is at least 80, and confidence values decrease or stay equal.

5. Hard: find protein domains

GET `/api/workshop/proteins/{protein_id}/domains/`

1. Find the protein in `DpcUniprotProtein`; missing ID → 404.
2. Query its DPCfam, DPCstruct, and Pfam mapping tables.
3. Order each mapping set by `id`; limit **each** to 100 rows.
4. Return protein ID, length, and the three lists. A known protein may have empty lists.

Checkpoint Use teaching ID `WSDEMO0001`. Its three lists should each contain 100 entries.

5. Solution: imports and protein lookup

Add these imports to `api/workshop_views.py`:

```
from dpc.models import DpcUniprotProtein, DpcUniref50Pfam
from dpcfam.models import DpcfamMcsSequence
from dpcstruct.models import DpcStructMcsSequence

@api_view(["GET"])
def protein_domains(request, protein_id):
    protein = get_object_or_404(
        DpcUniprotProtein,
        protein_id=protein_id,
    )
```

Keep the earlier imports and functions. Retrieve the protein first so that a missing ID returns 404 rather than three empty lists.

5. Solution: query the three mappings

Continue inside `protein_domains()`:

```
    dpcfam_domains = (
        DpcfamMcsSequence.objects.filter(protein=protein)
        .order_by("id")[:100]
    )
    dpcstruct_domains = (
        DpcStructMcsSequence.objects.filter(protein=protein)
        .order_by("id")[:100]
    )
    pfam_domains = (
        DpcUniref50Pfam.objects.filter(uniref50=protein)
        .order_by("id")[:100]
    )
```

All three relationships point to the same protein registry. The Pfam relationship is named `uniref50`; the other two are named `protein`.

5. Understand the response fields

List	Selected fields	Meaning
`dpcfam`	`mc_id`, `seq_range`	Family cluster ID and sequence interval.
`dpcstruct`	`mc_id`, `prot_range`	Structural cluster ID and protein interval.
`pfam`	`pfam_id`, `pfam_ranges`	Pfam accession and annotated interval.

- Both mapping models name their cluster relationship `mc`; `mc_id` exposes its stored ID.
- `.values(...)` selects dictionaries; `list(...)` collects them for the response.

5. Solution: return the three lists

Finish `protein_domains()`:

```
    return Response({
        "protein_id": protein.protein_id,
        "protein_length": protein.protein_length,
        "dpcfam": list(
            dpcfam_domains.values("mc_id", "seq_range")
        ),
        "dpcstruct": list(
            dpcstruct_domains.values("mc_id", "prot_range")
        ),
        "pfam": list(
            pfam_domains.values("pfam_id", "pfam_ranges")
        ),
    })
```

5. Solution: add the protein route

Extend the workshop import in `api/urls.py`:

```
from .workshop_views import (
    dpcfam_summary, dpcstruct_search, protein_domains,
)
```

Add inside the existing `urlpatterns`:

```
path(
    "workshop/proteins/<str:protein_id>/domains/",
    protein_domains,
    name="workshop-protein-domains",
),
```

5. Solution: empty versus missing

```
BASE=http://127.0.0.1:8001/api/workshop

# Expected: 200, with three empty lists
curl -i "$BASE/proteins/WSEMPTY0001/domains/"
# Expected: 404, because this protein does not exist
curl -i "$BASE/proteins/WSMISSING/domains/"
```

The existing protein returns:

```
{"protein_id": "WSEMPTY0001", "protein_length": 600,
 "dpcfam": [], "dpcstruct": [], "pfam": []}
```

An empty annotation list is a valid result. A missing protein is a different case.

5. Solution: check the 100-row limits

```
curl -sS "$BASE/proteins/WSDEMO0001/domains/" |
python3 -c '
import json
import sys
data = json.load(sys.stdin)
print("Protein:", data["protein_id"])
for key in ("dpcfam", "dpcstruct", "pfam"):
    print(key, len(data[key]))
'
```

Expected: `Protein: WSDEMO0001`, then `dpcfam 100`, `dpcstruct 100`, and `pfam 100`.

There are 130 mappings in each table. The response limit applies separately to each list, not to 100 combined entries.

6. Final test: prove the API works

Request	Expected
Known DPCfam summary	200
Missing DPCfam summary	404
POST to read-only summary	405
Structural search with pLDDT 120	400
Known protein domains	200
Missing protein domains	404

Checkpoint Also check ten sorted search results, three 100-entry mapping lists, and a known protein with no mappings.

6. Solution: final six status codes

Suppress long response bodies while checking statuses:

```
BASE=http://127.0.0.1:8001/api/workshop
status() {
  curl -sS -o /dev/null -w '%{http_code}\n' "$@"
}
status "$BASE/dpcfam/MC1/summary/"
status "$BASE/dpcfam/MC999999/summary/"
status -X POST "$BASE/dpcfam/MC1/summary/"
status "$BASE/dpcstruct/search/?min_plddt=120"
status "$BASE/proteins/WSDEMO0001/domains/"
status "$BASE/proteins/WSMISSING/domains/"
```

Expected, in order: `200, 404, 405, 400, 200, 404`.

If something does not work

- **Cannot connect:** check Django is running and use the port printed in its terminal.
- **Every new route is 404:** check imports and entries in `api/urls.py`, the `/api/workshop/` prefix, and the trailing slash.
- **HTTP 500:** read the traceback in the Django terminal; check model field names and indentation.
- **Database error:** use `workshop/manage.py`; the wrapper loads the teaching configuration.

6. Check your changed files

From the student project root:

```
git status --short
```

Expected after these exercises:

```
 M api/urls.py
?? api/workshop_views.py
```

The generated `.env.workshop`, virtual environment, and local logs stay ignored. Do not add them to Git.

You have built three scientific APIs

Retrieve, filter, and combine data with REST.

EASY Identify a metacluster by ID.

MEDIUM Search using validated filters.

HARD Combine three independent sets of protein mappings.

Student setup: branch `workshop/student-setup`, file `workshop/README.md`.

Open the workshop README on GitHub